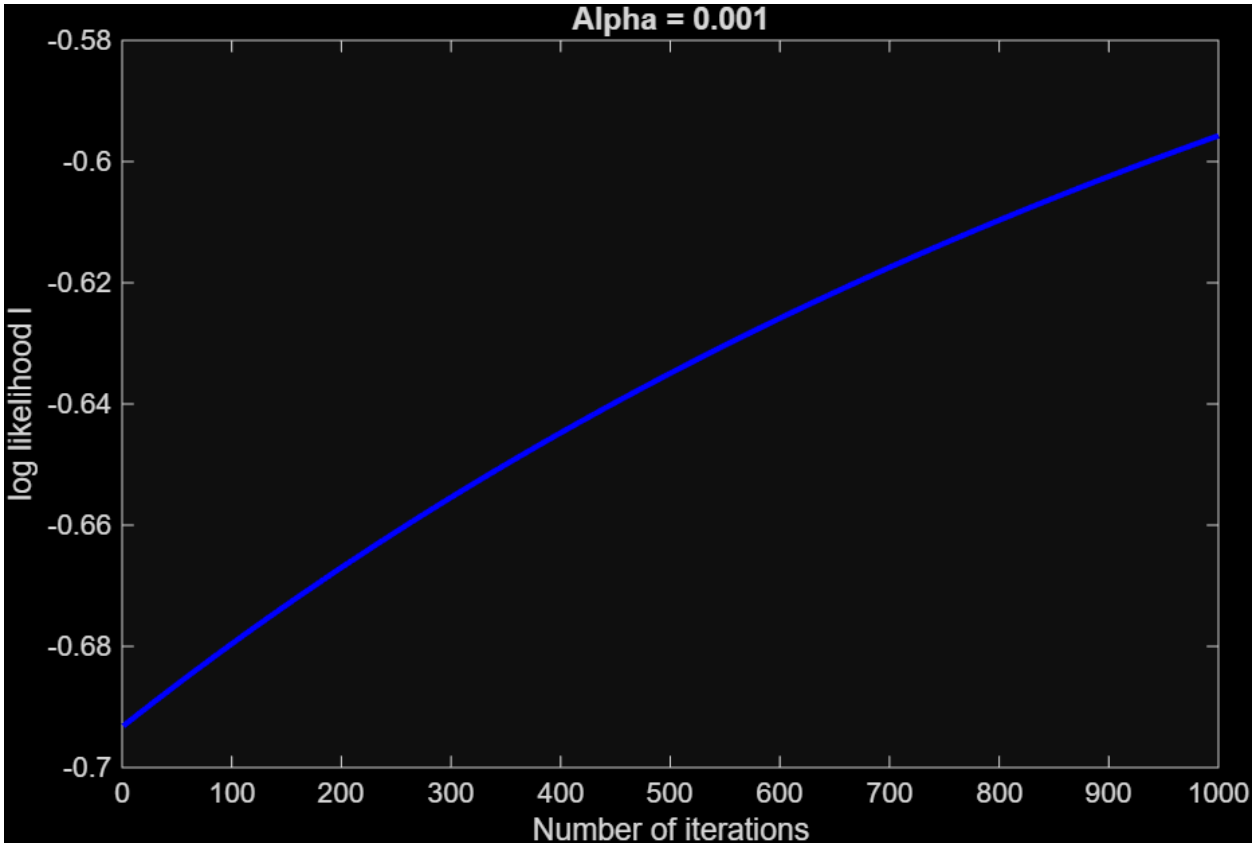
Nicholas Schwartz
October 14, 2026
CompE 510

Assignment M5 Report

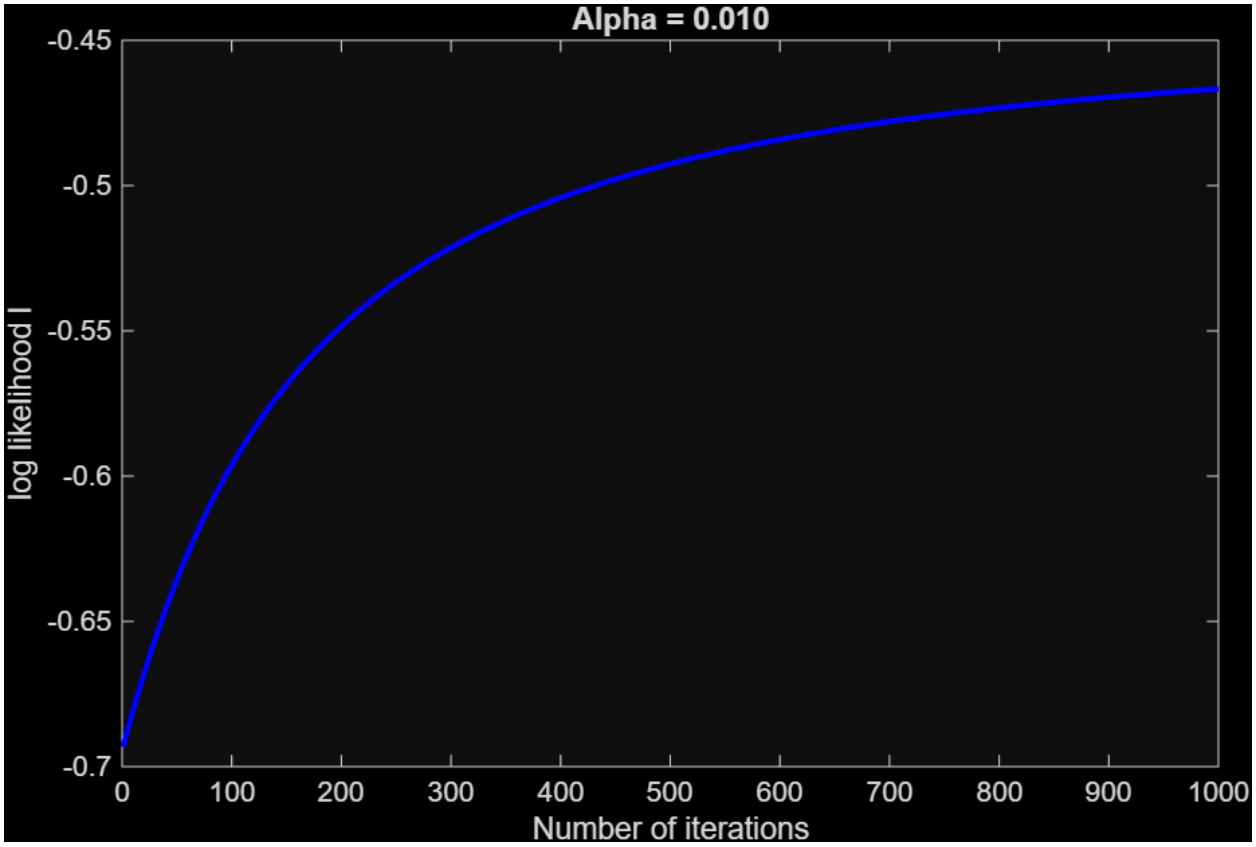
1. Run gradient ascent with at least three different learning rates. Show how the log likelihood function behaves over iterations. Based on the results, explain how the learning rate affects convergence and model performance. What learning rate worked best for your data, and why?

I tested gradient descent with five learning rates: alpha = 0.001, α = 0.1, a=1 and α = 7. They were all performed at 1000 iterations to give enough steps to see their behavior over time.

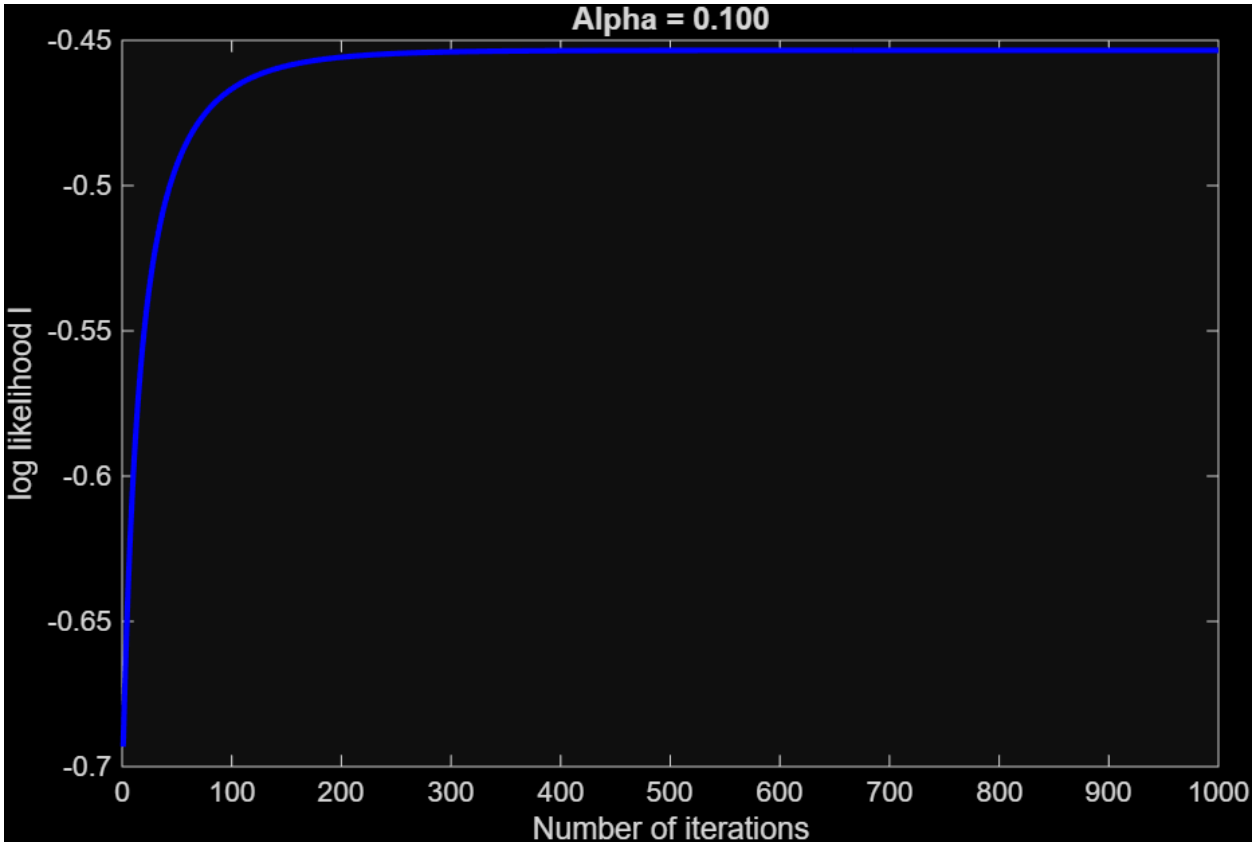
When alpha = 0.001, the log likelihood increased steadily but very slowly. This learning rate was stable, but the model would require many more iterations to reach the maximum log likelihood.

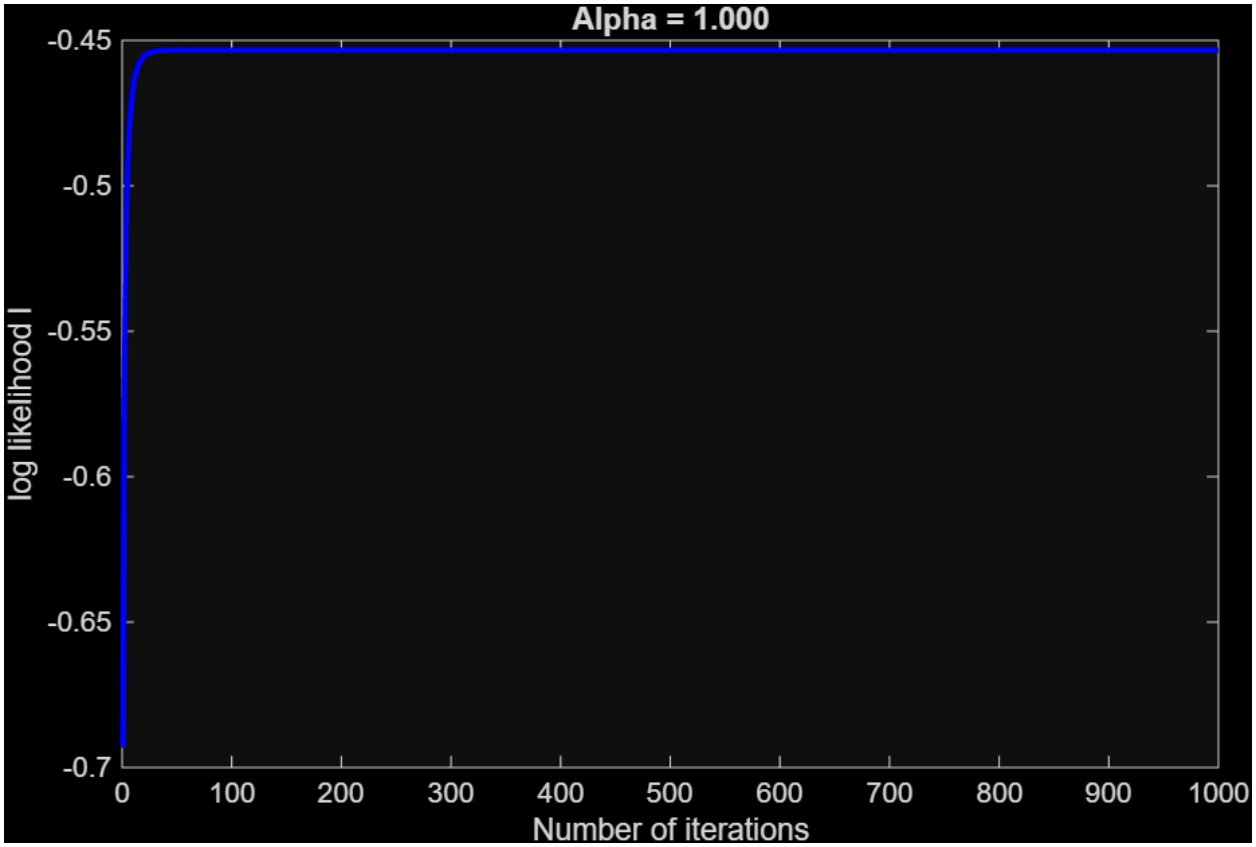


When alpha = 0.01, the log likelihood increased quicker but did not reach its maximum in the allotted time. This learning rate was stable, but the model would require many more iterations to reach the maximum log likelihood.

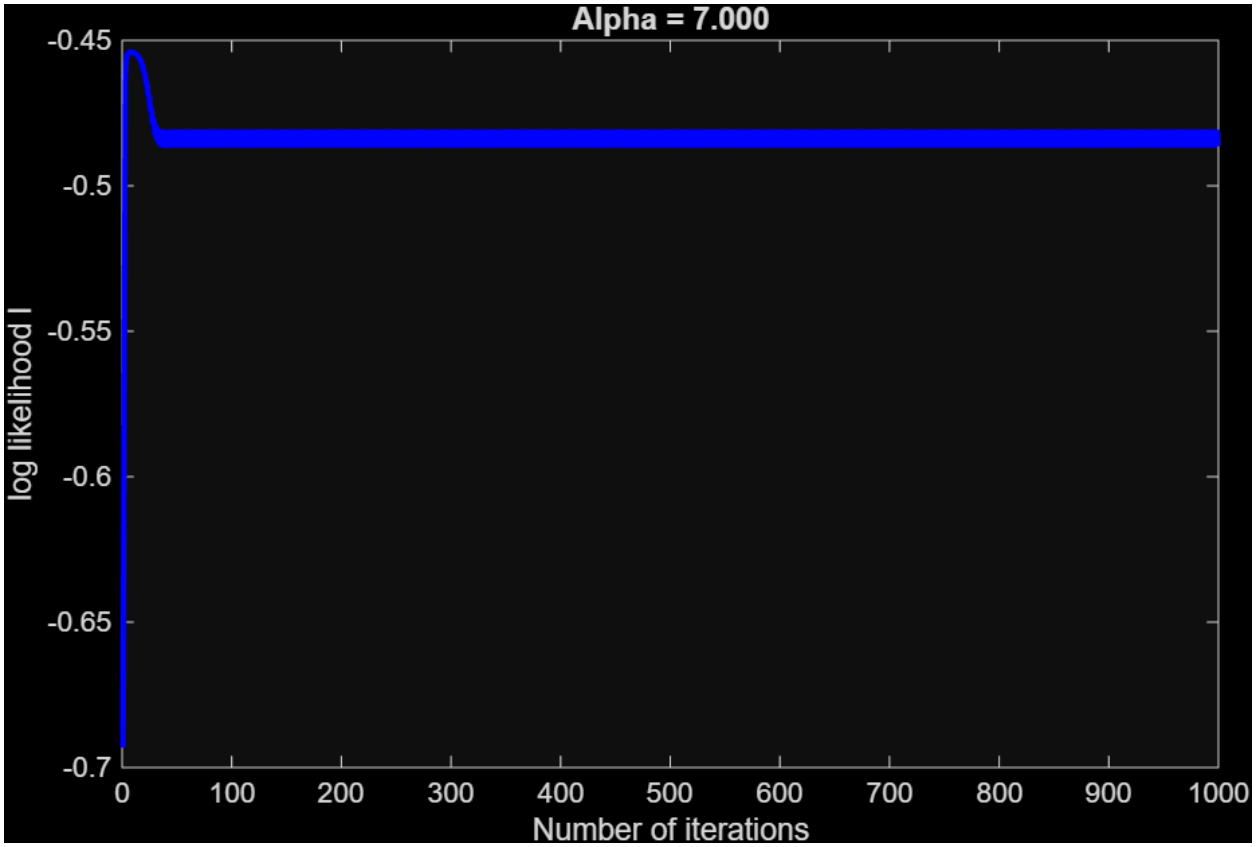


When alpha = 0.1 and alpha = 1, the behavior performed similarly. It grew to its maximum quickly and then leveled off well before the iteration count was reached. This is an effective range for the learning rate in this context. Both values have the same accuracy as they both converge to the same point after many iterations.


Alpha = 0.100
-0.45
-0.5
-0.55
-0.6
-0.65
-0.7
log likelihood l
0
100
200
300
400
500
600
700
800
900
1000
Number of iterations


Alpha = 1.000
-0.45
-0.5
-0.55
-0.6
-0.65
-0.7
log likelihood l
0
100
200
300
400
500
600
700
800
900
1000
Number of iterations

When alpha reached 7, the log likelihood varied dramatically from the other trials. Rather than reaching a maximum and then becoming stable, it reached the same maximum and then dropped to a lower value and remained unstable, shifting between values constantly.



The learning rate has a major effect on gradient descent. If the rate is too small, it makes training inefficient, and if a learning rate is too large, it can prevent convergence. Therefore a moderate rate around alpha = 0.1 is sufficient for this system.

As a result of these findings, I iterated around alpha =0.1 to find a sufficient learning rate and determined that alpha = 0.1 was sufficient to converge.I used a learning rate of **a = 0.1 and 500 iterations**.

2. In your own words, explain what the log-likelihood function represents in logistic regression. Why is it a more appropriate objective than minimizing squared error in this case?

The log likelihood represents the likelihood that the model's probability is correct. Each label y is treated as a Bernoulli variable where the likelihood comes from the sigmoid function. This keeps the probability between 0 and 1, a binary classification. The log likelihood is the log of the product of these probabilities over all the training examples. It measures the likelihood that each score is accurate per run and uses that to evaluate the beta. At the initial beta, the likelihood is a probability of 0.5 (l = ln(0.5) = -0.693) of being correct as there has been no training to influence

the result. This is represented as the beginning of the variable l_history. However by the end of the ascent, the likelihood l = -.486 leading to a probability of p=e^(-0.486) = 0.615. This shows that there is a higher likelihood of the patients having diabetes based on the training set.

The log likelihood is a more appropriate objective than the squared error as the result y is a binary result and the model outputs a probability. As it is a Bernoulli type of model, it is the more appropriate model compared to minimizing squared error. The squared error comes from the linear regression model which is a Gaussian model. This would not be appropriate in the case as there are only two possible verdicts, compared to a linear regression which could have any real number as an outcome. Because there are only 2 possible outcomes and that the labels are Bernoulli, the log likelihood is a more appropriate objective.

3. Logistic regression outputs probabilities. Select a few examples where the model was either very confident but incorrect, or uncertain but correct. What might cause such cases in real-world applications? How could you improve the model to handle these situations?

To determine the instances where it was confident and wrong or uncertain and correct, I utilized the sigmoid function on Xn_test*beta and predict on beta and Xn_test. I then compared the predicted answers to the correct values found in y_test to sort into the two categories. This is similar to what was instituted in the evaluateAccuracy function.

The model's confidence is a probability where values close to 0 or 1 are confident and those close to 0.5 are unconfident. I found that in some cases where the confidence was high that the patient was not diabetic but they were diabetic, there were missing values for the BMI, skin, and insulin with a high weight. Since the BMI was extremely low by this metric, it had a degree of certainty that they would not be diabetic but that is not the case since they actually had a high weight. I also found some that were the reverse, where the confidence was high that they were diabetic when they were not. In that case, the glucose and BMI were high and all signs pointed to them being diabetic. However, there are always factors outside of this list that could contribute to this. Diet, exercise, and family history are large contributors that are not accounted for. There was also missing data for several more of these examples.

For cases that were uncertain but the model was right, the same issues apply where there were multiple missing values in their row. The missing values influence the diagnosis by the regression and causes it to have less certainty that they are correct. Another was where most values were in the average range. Some patients can have almost no symptoms and still have diabetes, there are many other factors such as those previously mentioned that could influence it.

The missing values were what primarily caused the likelihood to report with low confidence and be right or high confidence with wrong diagnosis. The model could be improved by treating 0 values as the training median so as to not influence the result, thereby increasing the accuracy by effectively removing their influence. There is also the option of adding feature combinations

so that features are coupled together and the comparison can add to the calculation. Some features work in tandem, such as BMI and glucose, so it would be beneficial to see how those stack against each other while developing a diagnosis. One final option is to lower the threshold for the diagnosis. Rather than having it be p=0.5, it could be lowered so that there are more false positives rather than false negatives. It is much more detrimental to the patient to have a missed positive rather than a missed negative.

4. Describe what went well and what did not go well during the implementation. Also describe how the implementation could be improved.

I had difficulty implementing the likelihoodFunction. The difference between the dot product and matrix product tends to give me issues and will give dimensional errors when running the code. As the function names were the same as Assignment 4, I tried to utilize my code from the previous assignment only to find that it was incompatible with this assignment. I utilized AI to help troubleshoot my code when I ran into issues. It helped me discover that my predict function was compared to 0 rather than 0.5.

As for what went well, the beta values displayed data that made sense from the early phases. I performed checks on it as I went through to make sure that the values were making sense, as that was something I had trouble with on the previous assignment.